CSC3206 ARTIFICIAL INTELLIGENCE

ASSIGNMENT 2

NO.	MEMBER	STUDENT ID	CONTRIBUTION
1	Liang Yrenn Da	22067128	Code Implementation + Presentation
2	Ooi Jin Yon	22039069	Code Implementation + Explanation
3	Yeoh Jie Long	22071112	Code Explanation + Group Communication
4	Kavinesh Mardad	22033369	Problem Statement + Group Communication
5	Marcus Ong Xun	22019020	Result and Discussion + AI Transparency
6	Jimmy Teng Hau Ming	22047179	Result and Discussion + AI Transparency

Table of Content

1.0 Problem Statement 3
2.0 Code Explanation 4
2.1 Overview 4
2.2 State Space Representation 4
2.3 Goal Test 4
2.4 Cost Function 5
2.5 Heuristic function and admissibility 6
2.6 Search mechanics 7
2.7 Connectivity check 7
2.8 Output Generation 8
2.9 Modifications for Different Configurations 10
3.0 Result and Discussion 10
3.1 Scenario 1: Round Trip (Sunway University to Sunway University) 10
3.1.1 Scenario 1 Configuration 10
3.1.2 Result 11
3.1.3 Discussion 11
3.2 Scenario 2 (Sunway to no fixed end point) 13
3.2.1 Scenario 2 Configuration 13
3.2.2 Result 13
3.2.3 Discussion 13
4.0 Appendix 14
4.1 Code and Video Link 14
4.2 Group Discussion 15
4.3 AI usage transparency 16
4.4 Individual Reflection 17
4.4.1 Liang Yrenn Da 17
4.4.2 Ooi Jin Yon 18
4.4.3 Yeoh Jie Long 19
4.4.4 Kavinesh Mardad 20
4.4.5 Marcus Ong Xun 21
4.4.6 Jimmy Teng Hau Ming 21

1.0 Problem Statement

Route planning is a widely used application of Artificial Intelligence (AI), where search algorithms are utilised to determine the most efficient paths between locations in terms of the travel distance and travel time. In Assignment 1, the route planning problem was formulated as an AI search problem where three search algorithms; Depth-First Search (DFS), Breadth-First Search (BFS) and A* Search, were analysed and evaluated. A comparison of these algorithms led to the determination the A* search was the most suitable algorithm because of its ability to efficiently determine an optimal route by combining the actual travel cost with a heuristic estimate of the reaming cost.

The objective of the project is to implement an A* search algorithm to solve the route planning problem. The route begins in Sunway University and the goal is to visit all the residential locations of the group members at least once; the locations are represented as nodes in a weighted graph and the roads connecting the locations are represented as weighted edges based on the travel distance and estimated travel time collected from Google Maps. The A* search algorithm will be used to determine the most efficient route by considering the accumulated travel cost.

Assumptions:

- The travel distances and estimated travel time obtained from Google maps are assumed to be accurate throughout the journey.
- Traffic conditions are to remain constant during the entire route; no unexpected congestion or road closures are to occur.
- The same vehicle (car) is used throughout the journey. Factors such as vehicle performance and weather conditions are to be ignored.
- The heuristic function used by A* search algorithm is admissible and it does not overestimate the remaining travel cost.
- The graph is to be fully connected where every location is reachable from the starting point.
- Travel cost is based on the travel distance and travel time.

2.0 Code Explanation

2.1 Overview

The route planning scenario is a variant of the Traveling Salesperson Problem (TSP) in which the goal is to find a route that starts from a root node (Sunway University) and then visits a necessary subset of residential sites, ending at an optional end node. To solve this, the A* Search algorithm was implemented on a weighted, undirected graph. In lieu of a pointA to pointB search, the algorithm dynamically determines the best path to remove a "checklist" of needed nodes. The program is also intended to be very modular, so that START and END nodes can be set at run time without changing the data in the source file.

2.2 State Space Representation

In traditional pathfinding, a state represents one of the game's positions. However, our agent has to visit multiple specific targets, and the state has to have a "memory" of where he has been. Hence the state is represented by a tuple (current_location, frozenset(visited_residences)). A frozen set is strictly utilized instead of a standard list or set because the state needs to be immutable and hash able so it can be stored within the closed_list and the node_details dictionary. This ensures that being at "Jalan Templer" with zero houses visited is considered as a fundamentally different state than being at "Jalan Templer" with three houses visited.

The bookkeeping structure for each state

```
# ------------------------------------------------------------------------------
class LocationNode:
    def __init__(self):
        self.parent_state = None    # the state we came from
        self.g = float('inf')       # cost from start to this state
        self.h = 0                  # heuristic estimate to finish
        self.f = float('inf')       # g + h
```

Example of how the state tuple is generated dynamically during the search

```
new_visited = visited | ({location} if location in required else set())
new_state = (location, new_visited)
```

2.3 Goal Test

The goal test is used to determine whether the current state satisfies all the termination

criteria. It is implemented in the is_destination() function. The test tests for two conditions:

1. Are all necessary residential places added to the visited subset?
2. If the user defined a specific ending location, is the current_location equal to the end node? If both are true, the algorithm changes its state to success and sets up the path-tracing process.

```
def is_destination(location, visited, required, end):
    """
    Equivalent of is_destination(). Here the destination is reached only
    when every required residence has been visited AND (if an END location
    was requested) we are currently standing at END.
    """
    all_visited = set(required).issubset(visited)
    if end is None:
        return all_visited
    return all_visited and location == end
```

2.4 Cost Function

A significant consideration in the problem formulation was how to handle edge weights as the data contains both Travel Distance (km) and Travel Time (minutes). These measurements are on completely different scales and units. If we added them together it would give the larger number unfair control over the cost. To resolve this, a Min-Max Normalization approach was implemented in the edge_cost() function.

Distance and time are independently scaled down to a value between 0.0 and 1.0. The normalized values will multiply by an adjustable weight *(DISTANCE_WEIGHT = 0.5 and TIME_WEIGHT = 0.5)*

and summed together. This creates a unified and fair step cost ($g(n)$) that balances fuel efficiency (distance) and time efficiency.

```
DISTANCE_WEIGHT = 0.5
TIME_WEIGHT = 0.5

_all_d = [d for _, _, d, _ in state_space]
_all_t = [t for _, _, _, t in state_space]
MIN_D, MAX_D = min(_all_d), max(_all_d)
MIN_T, MAX_T = min(_all_t), max(_all_t)


def edge_cost(distance, time):
    norm_d = (distance - MIN_D) / (MAX_D - MIN_D) if MAX_D > MIN_D else 0.0
    norm_t = (time - MIN_T) / (MAX_T - MIN_T) if MAX_T > MIN_T else 0.0
    return DISTANCE_WEIGHT * norm_d + TIME_WEIGHT * norm_t
```

2.5 Heuristic function and admissibility

To maintain efficiency while finding the best path, the heuristic function must be admissible because it must never overestimate the true remaining cost. The *calculate_h_value()* function achieves this by using Dijkstra's algorithm *(dijkstra_all_pairs())* to precompute the true shortest path costs between all pairs of nodes on the map. During the A* search, the heuristic looks at the unvisited residences and returns the exact precomputed cost to the nearest unvisited target. If all targets are visited, it returns the cost to the final END node. This heuristic is strictly admissible because the cost to simply reach the closest remaining house will mathematically always be less than or equal to the actual cost required to visit *all* remaining houses.

```
def calculate_h_value(location, visited, required, end, shortest_cost):
    """
    Equivalent of calculate_h_value(). Instead of Euclidean distance to a
    single destination cell, this returns the shortest true graph-distance
    to the NEAREST unvisited residence (or to END once everything has been
    visited). This is a lower bound on the real remaining cost, so the
    heuristic stays admissible - it never overestimates.
    """
    remaining = [loc for loc in required if loc not in visited]
    if remaining:
        return min(shortest_cost[location][goal] for goal in remaining)
    if end is not None:
        return shortest_cost[location][end]
    return 0
```

2.6 Search Mechanics

The search mechanism utilizes a priority queue, implemented via Python's *heapq* library, to manage the *open_list*. The queue dynamically sorts the frontier based on the lowest $f(n)$ value *($f(n) = g(n) + h(n)$).* To prevent the search from getting stuck in infinite loops, a closed_list records every fully expanded state. Additionally, a custom LocationNode class acts as a bookkeeping structure for each state, storing the g, h, and f values, alongside a *parent_state pointer* which allows the trace *path()* function to walk backward from the goal state to the root to print the final ordered sequence.

```
    open_list = [(node_details[start_state].f, next(counter), start_state)]

    expansion_log = []

    while open_list:
        f, _, state = heapq.heappop(open_list)
        location, visited = state

        if state in closed_list:
            continue
        closed_list.add(state)

        new_visited = visited | ({location} if location in required else set())
        new_state = (location, new_visited)
        # keep node_details in sync for the (possibly) updated visited set
        if new_state not in node_details:
            node_details[new_state] = node_details[state]

        expansion_log.append((location, sorted(new_visited), round(node_details[state].g, 4),
                              round(node_details[state].h, 4), round(f, 4)))

        if is_destination(location, new_visited, required, end):
            print("The destination configuration is found (all residences visited"
                  + (f", ending at {end}" if end else "") + ").")
            path = trace_path(node_details, new_state)
            print("The Path is")
            print(" -> ".join(path))
            return path, node_details[new_state].g, expansion_log

        for neighbour, dist_km, time_min in graph[location]:
            step_cost = edge_cost(dist_km, time_min)
            g_new = node_details[new_state].g + step_cost
            neighbour_state = (neighbour, new_visited)

            if neighbour_state in closed_list:
                continue

            if neighbour_state not in node_details or g_new < node_details[neighbour_state].g - 1e-12:
                h_new = calculate_h_value(neighbour, new_visited, required, end, shortest_cost)
                f_new = g_new + h_new
                nd = node_details.get(neighbour_state, LocationNode())
                nd.g = g_new
                nd.h = h_new
                nd.f = f_new
                nd.parent_state = new_state
                node_details[neighbour_state] = nd
                heapq.heappush(open_list, (f_new, next(counter), neighbour_state))
```

2.7 Connectivity check

Before the A* search is permitted to run, a structural constraint check is enforced to guarantee that a solution is theoretically possible. The *check_connectivity()* function performs a BFS to

ensure the graph is fully connected. If a disconnected node is detected, the program will raises a value error rather than wasting compute time on an unsolvable path.

```python
def check_connectivity(graph, start):
    """
    Verify every location in the graph is reachable from `start` (BFS).
    This enforces the "graph is fully connected" assumption instead of
    just hoping the input data happens to satisfy it.
    """
    visited = {start}
    queue = [start]
    while queue:
        node = queue.pop()
        for neighbour, _, _ in graph.get(node, []):
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
    unreachable = set(graph.keys()) - visited
    if unreachable:
        raise ValueError(
            f"Graph is not fully connected from '{start}'. "
            f"Unreachable locations: {sorted(unreachable)}"
        )
    return True
```

2.8 Output Generation

To fulfil the visualization requirement, the *draw_route()* function utilizes the *networkx* library to build the structural relationships. Meanwhile, *matplotlib.pyplot* library to render the UI. An Agg backend is used to allow the script to execute headlessly and safely save the output as house_tour_route.png without crashing due to local display window errors. In addition, the code uses *FancyArrowPatch* with dynamically adjusted curved arrows to ensure that roads used more than once are displayed with separate curved paths. This prevents the route arrows from overlapping and improves the readability of the visualization.

```python
def draw_route(path, start, end, required, total_distance, total_time, total_cost, graph):
    """
    Renders the final route as house_tour_route.png:
      - the full road network in light grey
      - the chosen A* route highlighted in red, numbered in visit order
      - the start node in gold, an explicit END node (if set) in blue,
        and every other residence in white
    Repeated edges (a road used more than once, e.g. a location visited
    twice) are curved apart so their step-number labels never overlap.
    """
    Gnx = build_networkx_graph(graph)

    # FIXED DICTIONARY:
    # Arranged in a wide, spaced-out ring to prevent long edges
    # (like Kerinchi -> Bandar Puteri) from crossing other lines.
    pos = {
        'Jalan Kerinchi': (7, 6),
        'Jalan Templer': (3, 4),
        'Taman Kinrara': (4, 1),
        'Sunway University': (-1, 2),
        'Persiaran Tasik Barat': (-3, -1),
        'USJ 20': (-1, -5),
        'Bandar Puteri Puchong': (5, -4)
    }

    path_edges = list(zip(path, path[1:]))

    fig, ax = plt.subplots(figsize=(12, 9))

    # ---- base map: every road, thin and grey ----
    for u, v in Gnx.edges():
        x1, y1 = pos[u]
        x2, y2 = pos[v]
        ax.plot([x1, x2], [y1, y2], color='lightgray', linewidth=1.4, zorder=1)

    # ---- the chosen route: numbered arrows, curved apart when a road is
    #      reused (e.g. a location visited twice) so nothing overlaps ----
    edge_use_count = {}
    for step, (u, v) in enumerate(path_edges, start=1):

        # CRITICAL FIX: Use a directed tuple instead of a frozenset
        # so A->B and B->A are treated as separate visual arcs.
        key = (u, v)

        n = edge_use_count.get(key, 0)
        edge_use_count[key] = n + 1
        base_rad = 0.18 + 0.16 * (n // 2)
        rad = base_rad if n % 2 == 0 else -base_rad

        xy_a, xy_b = pos[u], pos[v]
        arrow = plt.matplotlib.patches.FancyArrowPatch(
            xy_a, xy_b, connectionstyle=f'arc3,rad={rad}',
            arrowstyle='-|>', mutation_scale=18, color='crimson',
            linewidth=2.2, zorder=2, shrinkA=18, shrinkB=18)
        ax.add_patch(arrow)
```

[...] Coordinates logic for numbered step badges

FIX: save using an absolute path and print it clearly

```python
save_path = os.path.abspath('house_tour_route.png')
plt.savefig(save_path, dpi=150, bbox_inches='tight')
print(f"\nSaved picture to: {save_path}")
```

2.9 Modifications for Different Configurations

Although the current implementation is designed for this route planning scenario, it can adapt to support different problem configurations by having some modifications. If the journey begins from a different starting location, the **START** location is already configurable at runtime through the *get_user_config()* function. Similarly, the implementation already supports routes that terminate at a specific destination. The goal test in *is_destination()* help to verify that all residential locations have been visited and that the current location matches the specified end node. Therefore, no additional modification is required for this configuration. Secondly, additional residential locations can be incorporated by adding new nodes and weighted edges to the *state_space* data structure. Since the implementation models the road network as a weighted graph, the A* Search algorithm can continue to operate without structural changes. However, The increase on number of locations will enlarge the search space. This result in higher execution time and memory consumption. Finally, the current implementation optimises a weighted combination of travel distance and travel time using the configurable *DISTANCE_WEIGHT* and *TIME_WEIGHT* parameters. If different optimisation objectives are required, the weighting parameters can be adjusted to place greater emphasis on travel distance or travel time without changing the underlying A* Search algorithm. This makes the implementation adaptable to different route planning requirements.

3.0 Result and Discussion

3.1 Scenario 1: Round Trip (Sunway University to Sunway University)

3.1.1 Scenario 1 Configuration

The start node of Sunway University is chosen as well as the end node using the A* Search algorithm. This algorithm will require the visit of all residential locations of all group members at least once before returning to the starting point.

3.1.2 Result

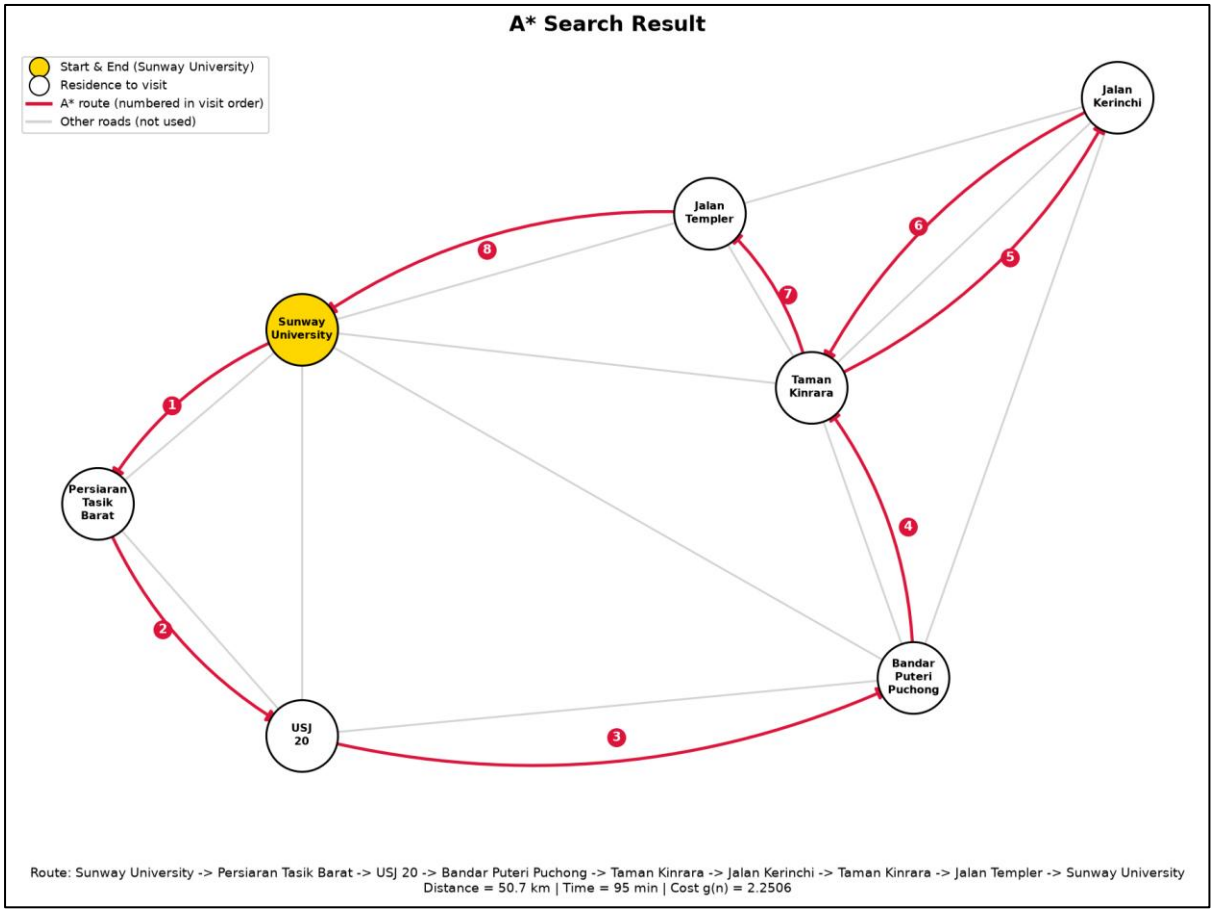


Figure 1. Search result for round tip starting and ending at Sunway University

Metric	Result
Start	Sunway University
End	Sunway University
Total Distance	50.7KM
Total Travel Time	95 minutes
Total Cost	2.2506
States Expanded	239

3.1.3 Discussion

Figure 1 shows the optimal route generated using the A* Search algorithm where the number arrows indicate the order which it is visited, starting from Sunway University and ending back at the same location again after visiting all other locations of required residence. The grey areas are alternative roads that are available but not used since they are more costly.

This route covers total travel distance of 50.7km with estimated time travel of 95 minutes. The algorithm also expanded 239 states before finding the optimal solution which is the one shown above with also the path cost of 2.2506 which is the combination of travel distance and travel time after the Min-Max normalisation and weighted cost calculation.

Taman Kinrara is visited twice as the road network is not fully connected to have direct link between all locations hence some locations will be visited twice, and, in this case, this return visit is deemed optimal in producing lowest overall travel cost. The optimal route travel is also shown below the image, and this implementation is successful as it fulfils the requirement of starting node and ending node as well as the need to visit all residence locations at least once while minimising the travel cost and finding the best optimal route.

3.2 Scenario 2 (Sunway University to no fixed end point)

3.2.1 Scenario 2 Configuration

This route started with Sunway University as the starting point and no fixed end point. A* has found the lowest cost route which visited all the six required residences.

3.2.2 Result

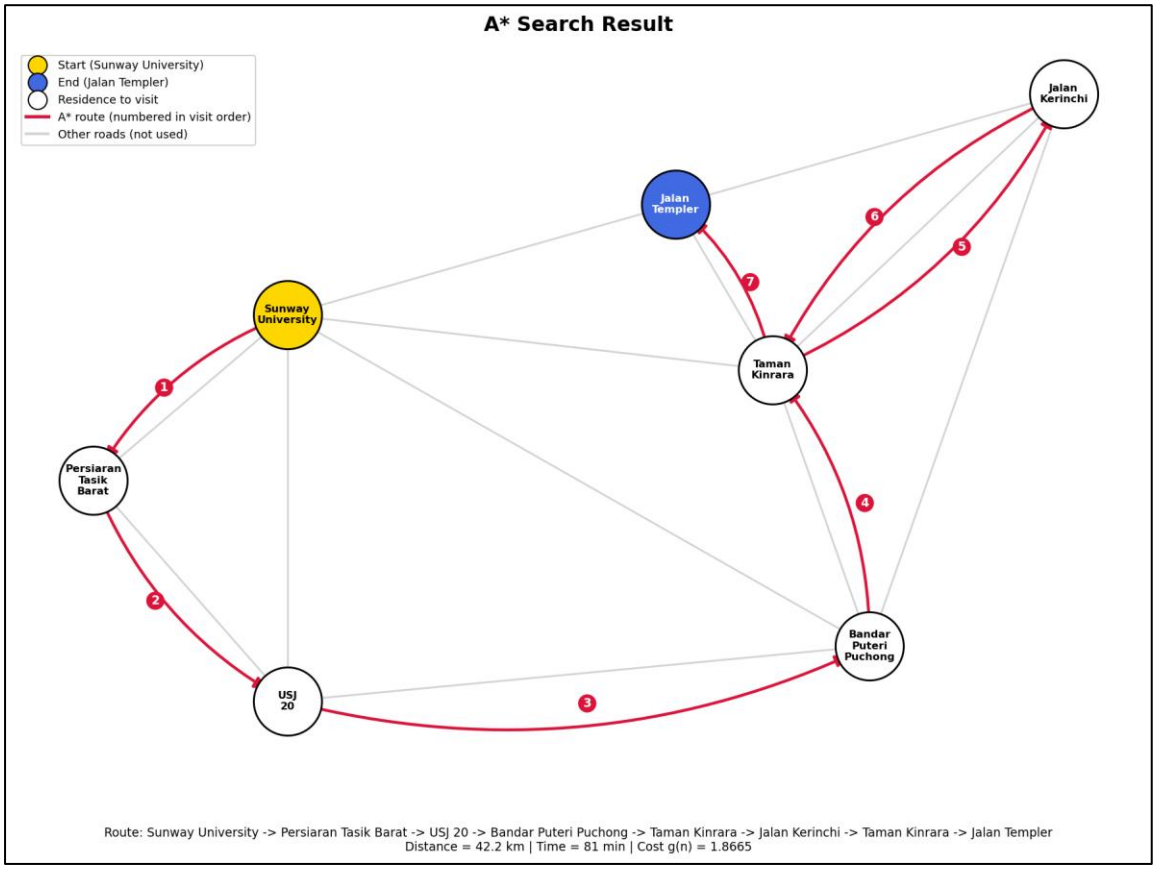


Figure 2: A* Search Result starting from Sunway University and end with Jalan Templer

Metric	Result
States expanded	174
Total Cost g(n)	1.8665 (normalised distance + time units)
Total Distance	42.2 km
Total Time	81 minutes

3.2.3 Discussion

For **Figure 2**, Taman Kinrara were being revisited because it is the only connection point to Jalan Kerinchi apart from the already-visited nodes. A* found that revisiting Taman Kinrara was cheaper than any other route. This indicates that the algorithm optimised the total route

cost rather than reducing unique stops.

The edge cost integrates normalised distance and time unit (0.5/0.5 weights). Therefore, the outcome reflects an equal balance between the two rather than a shortest-distance-only route.

The heuristic implements the true shortest-route cost computed by Dijkstra's algorithm to the nearest unvisited location. As a result, the optimal solution is guaranteed. Conversely, as each search state involves both current residence and the set of locations that have been visited, the search space becomes larger compared to a standard point-to-point search. This clarifies why the algorithm expanded 174 states regardless of generating a route with only 7 stops.

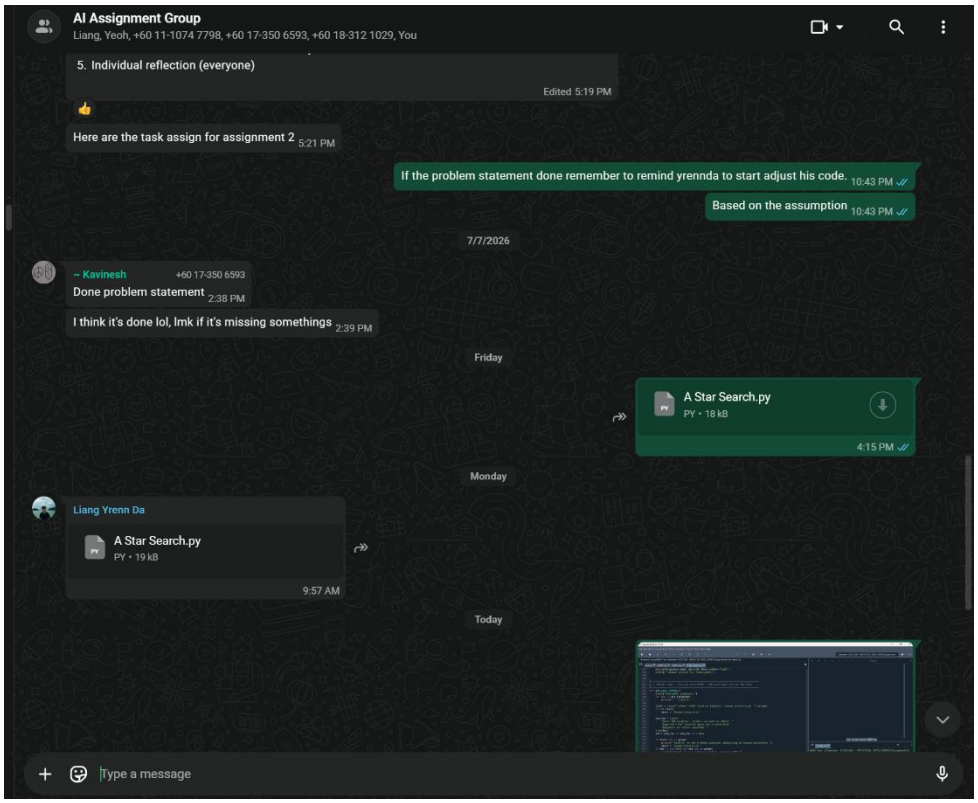
One limitation of this approach is that the route travelling cost depends on the static conditions and assumes constant traffic throughout the journey. As a result, the computed route is the only optimal under these fixed assumptions. Moreover, since there is no any specific endpoint, the algorithm selected Jalan Templer as the endpoint, as it is the lowest cost completion point, rather than it was explicitly requested by the user.

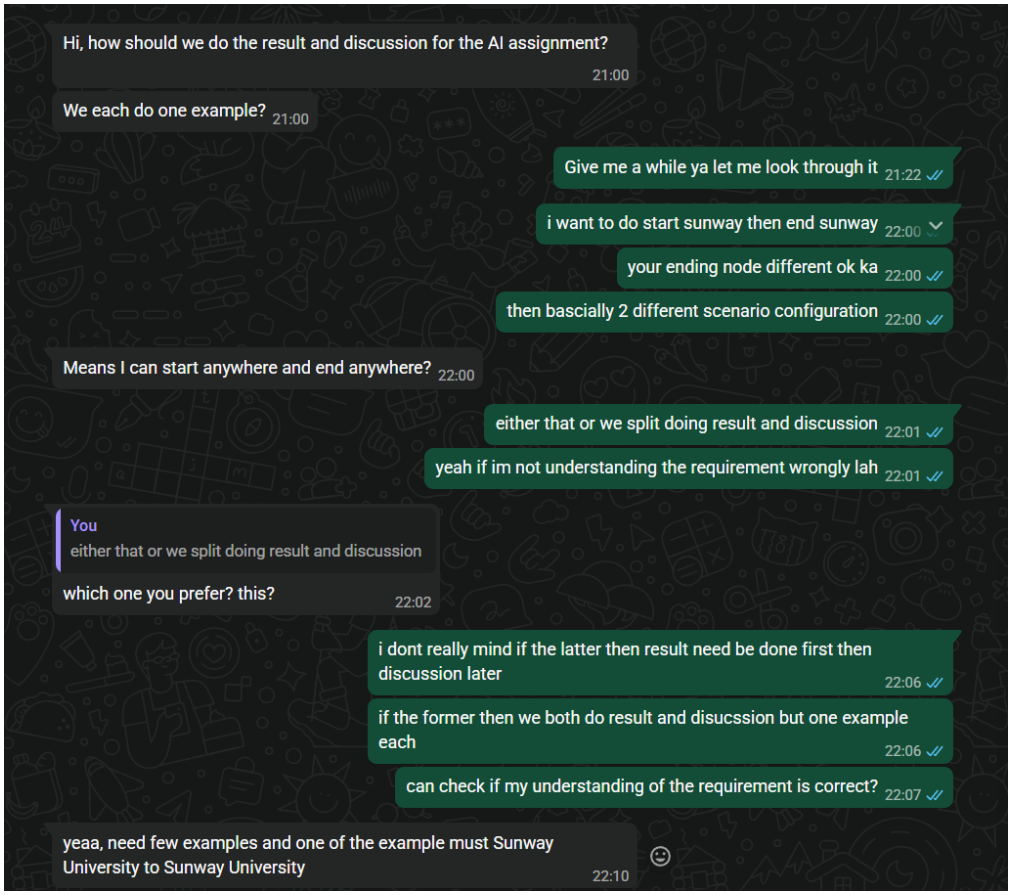
4.0 Appendix

4.1 Code and Video Link

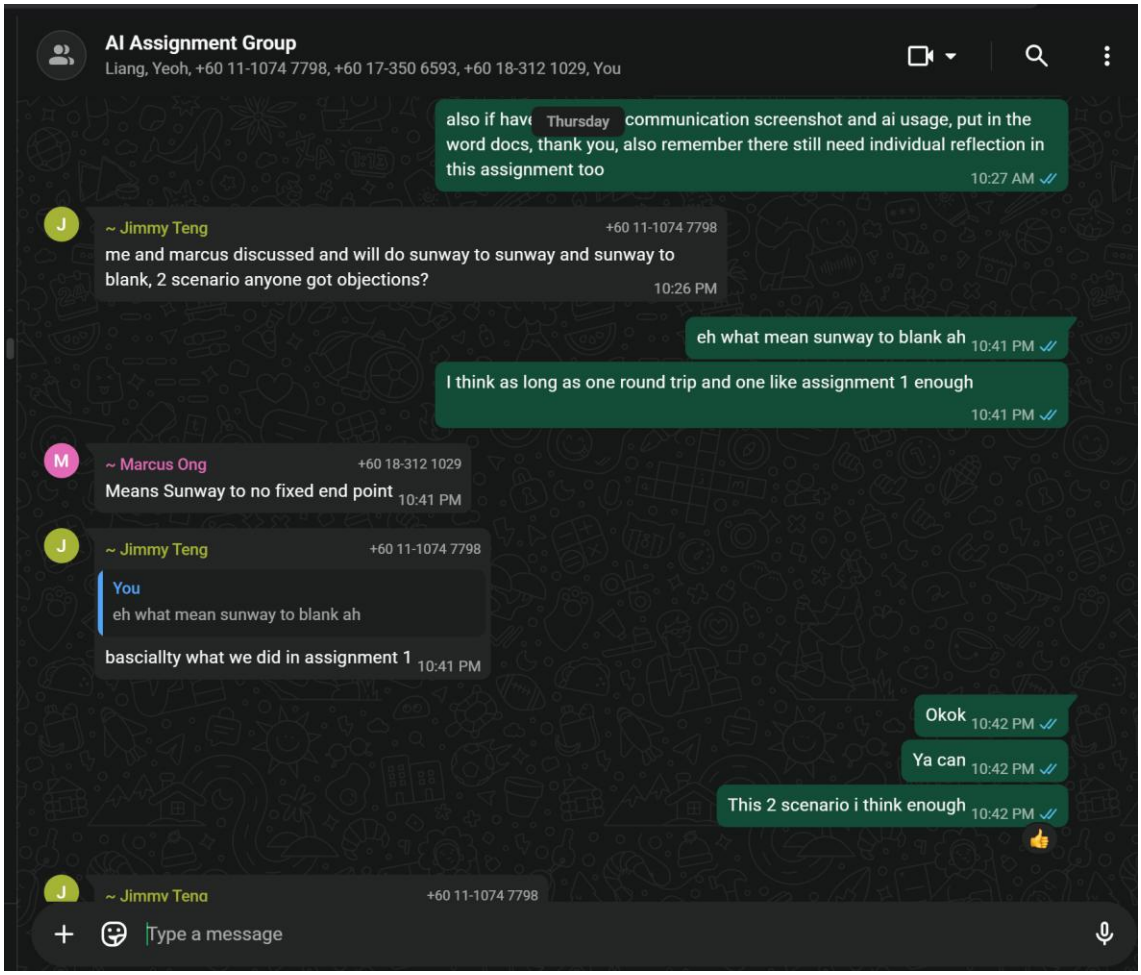
Code Link	
https://github.com/ItzSeir/Artificial-Intelligence-2-Assignment-Code	
Video Link	
YT Link:	https://youtu.be/sikqt3JIhnI
OneDrive Link:	https://imailsunwayedu-my.sharepoint.com/:v:/g/personal/22039069_imail_sunway_edu_my/IQCSBtnun8erQa5J8b6XzJpaAfyRjj-ZPxIXtjZULsY84NM?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJPbmVEcml2ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=j0Irqz

4.2 Group Discussion

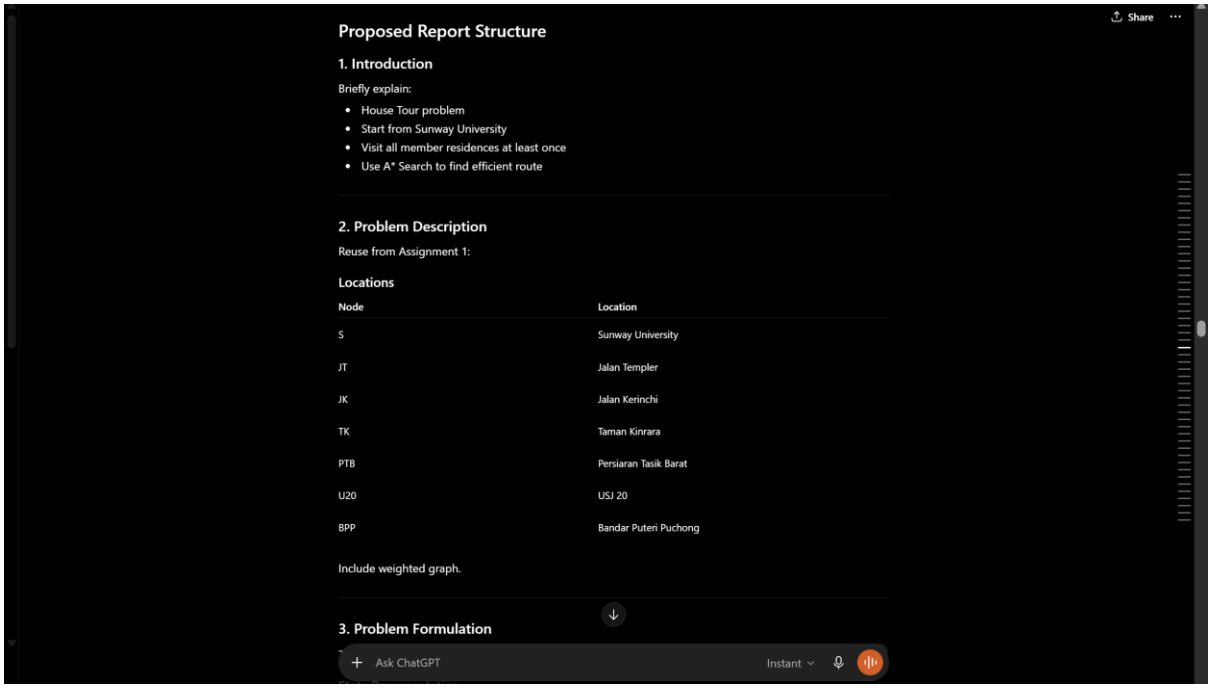


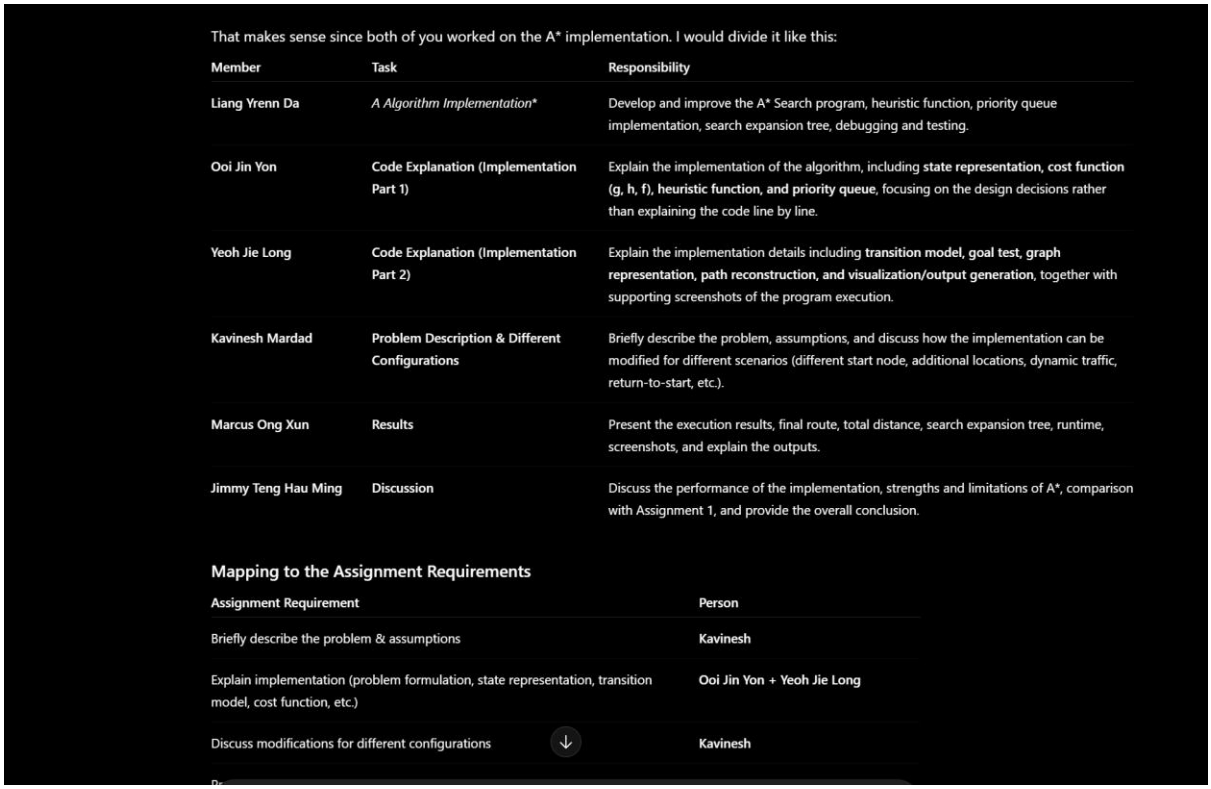


Discussion on how to split work and approach result and discussion section



4.3 AI usage transparency



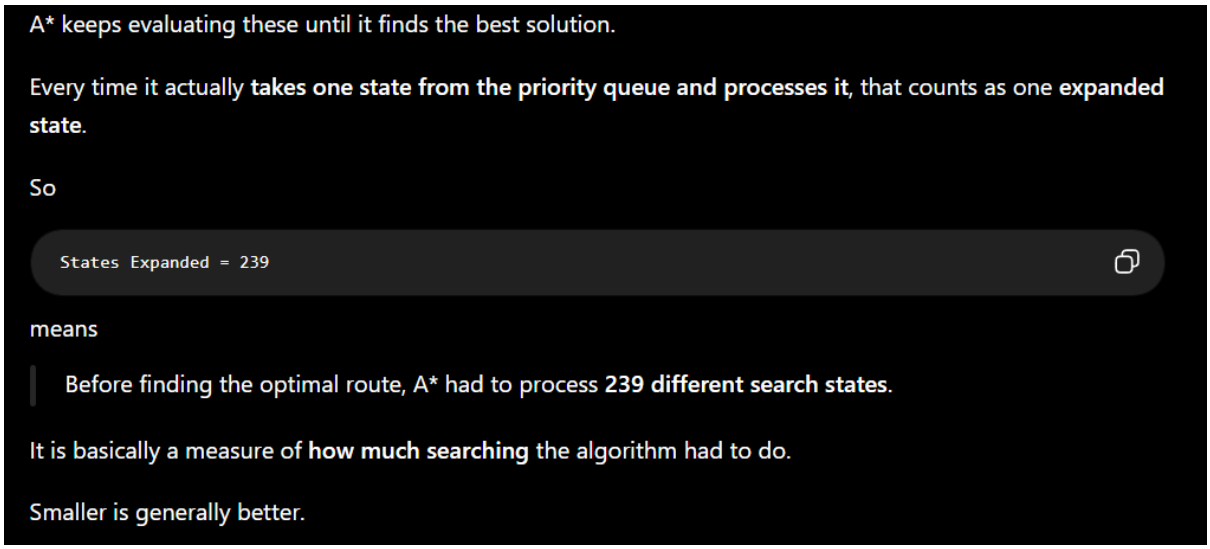


That makes sense since both of you worked on the A* implementation. I would divide it like this:

Member	Task	Responsibility
Liang Yrenn Da	*A Algorithm Implementation**	Develop and improve the A* Search program, heuristic function, priority queue implementation, search expansion tree, debugging and testing.
Ooi Jin Yon	Code Explanation (Implementation Part 1)	Explain the implementation of the algorithm, including **state representation, cost function (g, h, f), heuristic function, and priority queue**, focusing on the design decisions rather than explaining the code line by line.
Yeoh Jie Long	Code Explanation (Implementation Part 2)	Explain the implementation details including **transition model, goal test, graph representation, path reconstruction, and visualization/output generation**, together with supporting screenshots of the program execution.
Kavinesh Mardad	Problem Description & Different Configurations	Briefly describe the problem, assumptions, and discuss how the implementation can be modified for different scenarios (different start node, additional locations, dynamic traffic, return-to-start, etc.).
Marcus Ong Xun	Results	Present the execution results, final route, total distance, search expansion tree, runtime, screenshots, and explain the outputs.
Jimmy Teng Hau Ming	Discussion	Discuss the performance of the implementation, strengths and limitations of A*, comparison with Assignment 1, and provide the overall conclusion.

Mapping to the Assignment Requirements

Assignment Requirement	Person
Briefly describe the problem & assumptions	Kavinesh
Explain implementation (problem formulation, state representation, transition model, cost function, etc.)	Ooi Jin Yon + Yeoh Jie Long
Discuss modifications for different configurations	Kavinesh



A* keeps evaluating these until it finds the best solution.

Every time it actually **takes one state from the priority queue and processes it**, that counts as one **expanded state**.

So

```
States Expanded = 239
```

means

> Before finding the optimal route, A* had to process **239 different search states**.

It is basically a measure of **how much searching** the algorithm had to do.

Smaller is generally better.

Revise back the understanding of states expanded and its relation to assignment

4.4 Individual Reflection

4.4.1 Liang Yrenn Da

In this second assignment, my main task was to code the A* Search algorithm in Python and demonstrate the code's execution results. From the theoretical analysis of

assignment 1 to writing a fully functional, error-free script was a great learning experience.

The most significant technical challenge I encountered was when constructing the state space to accommodate a multiple-destination routing problem. As the algorithm was no longer a simple search from point-A to point-B, instead the algorithm needs to remember previously visited locations to complete the group's house tour. I overcame this by representing the state memory using an immutable frozen set, which allowed our priority queue to accurately track the changing path without overwriting the data. Additionally, balancing two completely different measurement units which are kilometers and minutes, required me to implement a min-max normalization cost function to ensure that neither unit of measurement unfairly biased the algorithm's decisions.

Furthermore, I had to ensure our heuristic was strictly admissible to guarantee the search remained optimal. By using Dijkstra’s algorithm, it is able to precompute the shortest path to the nearest unvisited residence. Thus, A* search could effectively eliminate inefficient paths without overestimating the remaining actual cost.

Lastly, the final code execution presentation helped me to effectively explain the concepts of backend logic. By organizing modular driver code, transforming models, and using automated graphical visualizations, I was able to bridge the gap between abstract software engineering and practical application. Overall, this project solidified my practical programming skills.

4.4.2 Ooi Jin Yon

This assignment allowed me to have a deeper understanding of how A* Search algorithm can be implemented to solve a route planning problem. Unlike the previous work that focused on comparing different search algorithms, this assignment needed us to make the algorithm into a working code and explain the implementation process behind it. My contribution in this assignment is helping my friend in code implementation and do the implementation explanation for the algorithm. By completing this assignment, I gained a

better understanding that developing an AI algorithm is not only about writing code but also setting up the problem correctly so it reflects a real-world situation. During the implementation, I noticed how a priority queue helps A * Search to expand the most promising nodes in an efficient way as per the evaluation function. I also had a better understanding of the importance of the choice of an admissible heuristic as it directly influences the ability of the algorithm in producing an optimal solution. By doing these in this assignment, it helps to enhance my understanding between theoretical concepts and practical implementation. Furthermore, I also improved my communication and teamwork skills during the whole assignment process with my teammates. We had discussed implementation ideas and reviewed each other's work to ensure the report was correct and align to the requirement. In short, this assignment helps to enhance my programming and analytical skills by let me know how AI search algorithms can be applied to solve real-world optimization problems.

4.4.3 Yeoh Jie Long

In the second assignment, my role was to explain the implementation of the A* search and support communication between groups, covering the transition model, the goal test, the graph representation, path reconstruction and visualisation. As this was my first time learning and applying A* Search, I initially found it challenging to understand that the algorithm does not evaluate only the current location, as the search state must also remember which residences have been visited. After that, I realized that one important idea is the use of a frozenset in the representation of the state. It is immutable and hashable, so every combination of the current location and whether houses have been visited can be stored and compared as a separate state. I realised that going to the same place after visiting different houses must be handled differently. I knew that the goal test was checking if all required residences had been visited, and when a target state was given, if the current one matched the target, and that the parent_state links were used to get the final route in the correct order. Furthermore, I had to grasp the rationale behind the implementation, not just the code, when explaining these concepts. I discussed with Yrennda and Jin Yon the search process, implementation decisions, explanation structure, and work distribution. We had several subjects and overlapping deadlines, so good coordination facilitated us to complete related

tasks efficiently. AI tools also helped in coding and selecting appropriate technical terms for the report. I checked the outputs by running the code, looking at the route diagrams created and confirming they matched the assignment and the concepts used in A*. Lastly, I would expand the project by testing more route configurations and using dynamic traffic data rather than static traffic data the current system uses.

4.4.4 Kavinesh Mardad

For this Assignment 2, my main contribution was towards the Problem Statement of the report. The responsibility of clearly describing the route planning problem and explaining the object of the implementation of A* search algorithm is what I worked on. Moreover, I also had to define the assumptions that formed the basis of the implementation. It was important to ensure that the problem statement included a clear connection with the analysis done in Assignment 1 and that it meets the implementation requirements in Assignment 2. For instance, explaining why A* search was chosen and how the route planning scenario will be represented using nodes, weighted edges and travel costs.

This assignment enabled me to strengthen my understanding of the importance of clearly defining a problem before developing its solution. I realized that defining reasonable assumptions (such as constant traffic conditions and an admissible heuristic function) will help model real world problems that will in return justify the suitability of AI search techniques.

One challenge that I faced was making sure the problem statement was concise but still included all the necessary information to support the implementation. To overcome this issue, I referred to the work completed in the previous assignment (Assignment 1) and discussed the content with my group members to sustain the accuracy of the descriptions.

As a group, we ensured to maintain regular communication so that the assignment's progress can be monitored. Furthermore, we were able to collaborate effectively through the project by dividing up the tasks based on each member's strengths. By maintaining regular discussions, we were able to review each other's work and share the necessary feedback so that our report was accurate and consistent with the requirements.

4.4.5 Marcus Ong Xun

In this assessment, I was in charged to implement and testing the A* search for the hour tour route. I was assigned to complete the results and discussion for scenario 2 based on what the Python code produced.

One of the challenges I faced was making sense of the route revisiting Taman Kinrara twice. My first thought was that this should be a mistake, so I spent much time going back over the route connection before realising that the algorithm is actually right. This experience helped me to understand that the result that I did not expect is not considered failed or wrong. I should understand the reason behind the outcome before assuming that is is an error.

Working with my groupmates really helped me to understand this assignment better. We compared and discussed our individual tasks, which made me notice the things I missed on my own.

In this assignment, I used AI mainly to help me understand the assignment requirements, organise my thoughts, and write things in a clearer manner. All the problem-solving, understanding the result, and debugging were done by me.

Overall, this assignment helped me to gain a better understanding of how A* search works in real-life, and it gave me more confidence in explaining technical ideas.

4.4.6 Jimmy Teng Hau Ming

For this second assignment, I am tasked with result and discussion part where I work on one of the scenario examples of our current work with involves the use of A* search algorithm. These two assignments help to strengthen my understanding and how AI search algorithms can be used to apply in solving real world issues. This assignment makes me have a more hands on experience rather than just understanding of theory from lecture notes.

Being able to interpret outputs such as total travel cost and number of states expanded help me understand and appreciate search performance and computational efficiency.

Working in a group also continue to help me build and improve my communication and collaboration skills. I learned the importance of discussing ideas and managing work while at the end combining our contributions together into a proper report instead of just working separately alone without communication.

AI tools are useful for me as it helps me in revising and reviewing my understanding on how things work while filling in the knowledge gap. However, I also do know that AI generated answer should not be accepted on its own without verification hence I still am able to learn more as I compare explanation and answers across different sources and understand better. This experience further reinforces the idea that AI should only be learning assistant rather than replacement for human thinking.